

Recap

Design Patterns P1

Proven Solutions to Common Problems

Design Pattern: What?

- A **reusable solution** to a **commonly occurring problem** in software design
- **Not** finished code you can copy paste it
- A **Template** or **Blueprint** for solving a problem
- A **Description** of **how** to solve a problem

Design Pattern: Why?

- **Proven Solutions**
 - tested by thousands of developers
- **Faster Development**
 - You don't have to reinvent the wheel!
- **Avoid Common Mistakes**
 - Learn from others' experience
- **Best Practice**
 - It came from years of experience and wisdom
- **Common Vocabulary**
 - "Let's use a **Factory** here" , Everyone understands!
- **Maintainable Code**
 - Easier to understand, modify, extend

Design Pattern: Components

- **Pattern Name**
 - A Short Descriptive Name to Easier Communication
- **Problem**
 - When to apply the pattern?
 - The context and conditions
- **Solution**
 - The general structure of how the pattern solves the problem
- **Consequences**
 - Trade-offs and results
 - Pros & Cons

Design Patterns: When do you think about it?

- **Requirements Guide the Need for Patterns**
 - Requirements → Identify Problems → Problems Suggest Patterns
- **Based on problems and constraints in the requirements, such as:**
 - Need for flexibility
 - Need to change behaviors dynamically
 - Need to manage complex object creation
 - Need to reduce coupling
 - Need for scalability or extensibility
- **Remember**
 - Design Decision is made during **Design Phases**, Not in requirements gathering and analysis phases

Design Patterns: When to use?

Use When

- Problem is recurring
- Need flexibility for future changes
- Building large scale applications
- Code need to be maintainable lone-term

Don't Use When

- Problem is **unique**/one time
- Simple solution works fine (prefer **KISS**)
- Over-engineering would result
- Performance is critical
(some patterns add overhead)

Design Patterns: Types

GoF Patterns

- Introduced in 1994 (“**Gang of Four**” book)
- **Classic** 23 patterns
- **Focus: OOP**, object **creation**, **composition** and **communication**
- **Categories**
 - Creational
 - Structural
 - Behavioral

Modern Patterns

- **Patterns** for cloud, distributed systems, microservices, and DevOps
- **Focus: scalability, resilience, deployment, communication, data consistency**
- **Categories**
 - Architectural patterns
 - Distributed system patterns
 - Cloud-native & microservices patterns

Modern Patterns: Examples

- Microservices Architecture
- API Gateway Pattern
- Circuit Breaker Pattern
- Event Sourcing
- Saga Pattern
- Strangler Fig Pattern

GoF Categories

- **Creational Patterns**

- Object **Creation**
- How should objects be **Created**?
- 5 Patterns

- **Structural Patterns**

- Object **Composition & Relationships**
- How should objects be **Composed**?
- 7 Patterns

- **Behavioural Patterns**

- Object **Communication & Responsibilities**
- How should objects be **Interact**?
- 11 Patterns

GoF Design Patterns

Creational

- Singleton
- Factory
- Abstract Factory
- Builder
- Prototype

Structural

- Adapter
- Facade
- Decorator
- Proxy
- Composite
- Bridge
- Flyweight

Behavioural

- Observer
- Strategy
- Template
- Visitor
- Iterator
- Memento
- Command
- Interpreter
- Mediator
- Chain of Res.
- State

Design Patterns: *Motivations*

- **Java SDK**

`java.util.Iterator`



Iterator Pattern

`Runtime.getRuntime()`



Singleton Pattern

`StringBuilder`



Builder Pattern

- **Spring Framework**

`BeanFactory`



Factory Pattern

`ApplicationContext`



Singleton Pattern

`AOP`



Proxy Pattern

`JdbcTemplate`



Template Pattern

Creational Design Patterns

- How should objects be **Created**?
 - Object creation mechanisms
 - Encapsulating the creation logic
 - Improving flexibility and reducing coupling
- **Help to avoid:**
 - Complex creation logic spread across the code
 - Tight coupling between classes
- **Solve creation problems like:**
 - Reusing the same object (**Singleton**)
 - Creating families of related objects (**Abstract Factory**)
 - Building complex objects step-by-step (**Builder**)
 - Deferring creation until necessary (**Lazy creation**)

Singleton Pattern

*Ensure a class has Only One Instance and provide a
Global Point Of Access to it*

Challenges

- **Resource Overhead**

- How do we prevent heavy resources (e.g., **DB connection, logging files**) from being exhausted by creating too many identical objects?

- **Inconsistent State**

- If different modules create their own stateful objects (**cache, config store**), how do we keep **data consistent** across the app?
- What stops Module A and Module B from holding **outdated versions**?

- **Global Access Without Global Variables**

- How do we share an object across the entire application **without** using dangerous **global variables** that can be accidentally overwritten?

How to Fix?

- **Restricting Creation**

- How can we stop other classes from using `new ClassName()` whenever they want?

- **Storing the Instance**

- If we can't create the object from the outside, where does the object live?

- **Accessing the Instance**

- How do clients actually get the object if the constructor is locked?

How to Fix?

- **Restricting Creation**

- How can we stop other classes from using new ClassName() whenever they want?
 - make the constructor **private**

- **Storing the Instance**

- If we can't create the object from the outside, where does the object live?
 - create it as private static object inside same class

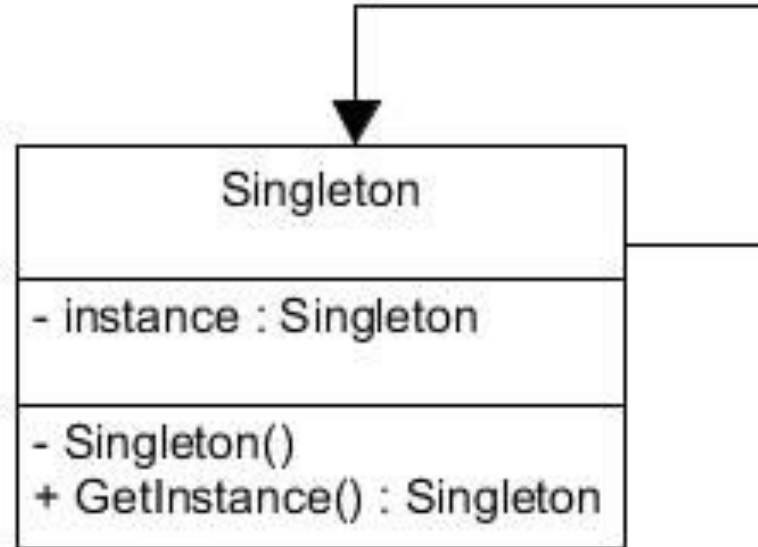
- **Accessing the Instance**

- How do clients actually get the object if the constructor is locked?
 - make the public static method to get the static instance you made

Result

- You are guaranteed that `Instance A == Instance B` is always true. You have strict control over the Singleton instance

Singleton: *Class Diagram*



Hands On Time

Implementation Types

Singleton: *Use Cases*

- **Database Connection**
 - Connection pooling, resource management
 - One pool serves all requests
- **Logger**
 - Centralized logging, consistent output
 - All classes log to same destination
- **Configuration Manager**
 - One source of truth for settings
 - Avoid loading config multiple times
- **Cache Manager**
 - Shared cache across application
 - Consistent data, memory efficiency

Singleton Pattern: *Java Examples*

- **JDK**

- java.lang.Runtime
- java.awt.Desktop

- **Spring Framework**

- Beans are singleton (per container) by default
- ApplicationContext Class

- **Logging Libraries**

- Log4j
- SLF4J

Trade-off: Convenience vs Coupling

- **Gain**

- You get easy, instant access to the object from anywhere in your code
(Singleton.getInstance().doSomething())

- **Loss**

- Your code becomes tightly coupled to the Singleton class
- You cannot easily replace it with a different implementation

Trade-off: Efficiency vs Complexity

- **Gain**

- You save memory and initialization time (Lazy Loading)

- **Loss**

- You introduce **threading complexity**
- In a **multi-threaded** environment (like a web server), you must implement locking/synchronization to prevent two threads from creating two "Singletons" at the exact same time

Singleton Pattern: Pros (in true concern)

- **Controlled Access**
 - Single point of access to the instance
 - Encapsulates how instance is created
- **Reduced Memory Usage**
 - Only one instance in memory
 - No duplicate objects
- **Global State Management**
 - Consistent state across application
 - All clients see same data
- **Lazy Initialization (Optional)**
 - Create instance only when needed
 - Save resources if never used
- **Flexibility**
 - Easy to change to allow multiple instances later
 - Subclassing is possible

Singleton Pattern: Cons

- **Tight Coupling**
 - Classes depend on singleton directly
 - Violates Dependency Inversion Principle
- **Violates SRP**
 - Class manages itself AND its lifecycle
- **Multithreading Complexity**
 - Need careful implementation for thread safety
 - Synchronization overhead
- **Testing Difficultes**
 - Hard to mock in unit tests
 - State persists between tests

Singleton Pattern: Summary

- **Problem**

- Need exactly one instance of a class
- Need global access to that instance
- Want to control instantiation

- **Solution**

- Private constructor
- Static instance holder
- Static getInstance() method
- Thread-safe implementation

- **Consequences**

- Controlled access, memory efficient, global state
- Testing difficulties, tight coupling, increasing complexity, violate SRP & DI

Builder Pattern

separates object construction from its representation

Challenges

- **The Telescoping Constructor**

- What happens when a class has MANY PARAMETERS, some required and some optional?
 - for n parameters, you need up to 2^n constructors!

- **Confusing Parameter Order**

- How do you know which parameter is which when they have the same type?

- **Null Values Everywhere**

- How do you skip optional parameters when using constructors?

- **Complex Object Construction**

- How do you construct objects that require Multiple Steps or Conditional logic during construction?

Solution: *Builder Pattern*

- Separate the **Construction** of a **complex object** from its **Representation** so that the **same construction** process can create **different representations**
- **It is about:**
 - **Step-by-Step Construction**
 - Build objects incrementally (set one attribute at a time)
 - **Separation of Concerns**
 - Construction logic separate from object itself
 - **Builder** knows **How** to build
 - **Product** is **What** gets built
 - **Fluent API**
 - **Method chaining** for readability (`builder.setA().setB().setC().build()`)
 - **Immutable Objects**
 - Create immutable objects easily
 - All properties set before object is created

Hands On Time

Builder Pattern: Pros

- **Eliminates Telescoping Constructors**
 - One **builder** handles all **parameter combinations**
 - No constructor **explosion**
- **Fluent, Readable API**
 - Method chaining reads like English (**Self-documenting code**)
- **Easy Optional Parameter**
 - No null passing required (Just skip it)
- **Separation Of Concerns**
 - Construction logic isolated in Builder
- **Validation at Build Time**
 - Invalid objects cannot be created
 - Fail fast with clear messages

Builder Pattern: Cons

- **More Code**
 - Need to Create Builder class and duplicate fields, and setter functions
 - **But** you can use **Lombok @Builder** in Java
- **Object Creation Overhead**
 - Builder object created then discarded
 - Extra memory allocation
 - **But** it's not a big problem
- **Overkill for simple objects**
 - Not needed for classes with few parameters
 - Unnecessary complexity for simple cases
 - **Tip:** use it only when you have 4+ parameters

Builder Pattern: *Java Examples*

```
new StringBuilder()  
    .append("Hello ")  
    .append("World")  
    .toString();
```

```
DateTimeFormatter formatter = new DateTimeFormatterBuilder()  
    .appendPattern("dd-MM-yyyy")  
    .toFormatter();
```

```
HttpRequest request = HttpRequest.newBuilder()  
    .uri(URI.create("https://example.com"))  
    .GET()  
    .build();
```

```
http  
    .csrf().disable()  
    .authorizeRequests()  
    .anyRequest().authenticated();
```

```
WebClient client = WebClient.builder()  
    .baseUrl("https://api.example.com")  
    .defaultHeader("Accept", "application/json")  
    .build();
```

```
RequestOptions options = RequestOptions.builder()  
    .setApiKey("sk_test_123")  
    .setIdempotencyKey("unique-operation-1")  
    .build();
```